5.6 Context

Управление временем жизни операций, отмена и передача данных запроса.

- Контекст передаётся через границы функций.
- Операции сами реагируют на сигнал отмены.
- В примерах Cause и *Cause требуется Go 1.21 или новее для всего набора функций.

Отмена не останавливает код принудительно

- cancel сообщает, что результат операции больше не нужен.
- Горутина должна проверить ctx.Done() или вызвать API, поддерживающий контекст.
- После возврата main завершается вся программа: остальные горутины не продолжают работать.
- Для корректного завершения подайте сигнал отмены и дождитесь работников через канал результатов или sync.WaitGroup.

Четыре метода Context

Сигнатуры методов; time.Time из пакета time

```
type Context interface {
    Deadline() (time.Time, bool)
    Done() <-chan struct{}
    Err() error
    Value(key any) any
}
```

- Deadline сообщает границу времени; bool показывает, задана ли она.
- Done закрывается при отмене. Err объясняет отмену.
- Value возвращает данные по ключу или nil, если значения нет.

Background и TODO

Фрагмент функции; import "context"

```
root := context.Background()
pending := context.TODO()
```

- Оба возвращают контекст, не равный nil, без дедлайна, значений и собственного сигнала отмены.
- Background используют как корень контекста приложения или теста.
- TODO отмечает место, где подходящий контекст пока не определён.
- Вниз по цепочке вызовов передавайте полученный ctx, а не новый Background.

WithCancel: ручная отмена

Фрагмент функции; parent - входной context.Context

```
ctx, cancel := context.WithCancel(parent)
defer cancel()

// Когда операция больше не нужна:
cancel()
<-ctx.Done()
fmt.Println(ctx.Err()) // context canceled
```

- cancel можно вызывать повторно.
- Отмена родительского контекста передаётся дочернему; отмена дочернего не отменяет родителя.

Работник должен реагировать на отмену

Функция; импорты context и time

```
func wait(ctx context.Context, d time.Duration) error {
    timer := time.NewTimer(d)
    defer timer.Stop()
    select {
    case <-ctx.Done():
        return ctx.Err()
    case <-timer.C:
        return nil
    }
}
```

Для долгого вычисления проверяйте отмену между этапами. Сам по себе context не прерывает произвольную функцию.

WithDeadline: конкретный момент времени

Фрагмент с функцией wait из предыдущего слайда

```
deadline := time.Now().Add(3 * time.Second)
ctx, cancel := context.WithDeadline(parent, deadline)
defer cancel()

if err := wait(ctx, 5*time.Second); err != nil {
    fmt.Println(err)
}
```

- Контекст отменится по дедлайну, при отмене родителя или при вызове cancel - что наступит раньше.
- Вызов cancel освобождает связанные ресурсы. Он не означает ожидание завершения работника.

WithTimeout: ограничение длительности

Фрагмент; импорты context, time, errors и fmt

```
ctx, cancel := context.WithTimeout(parent, time.Second)
defer cancel()

err := wait(ctx, 5*time.Second)
if errors.Is(err, context.DeadlineExceeded) {
    fmt.Println("Лимит времени исчерпан")
}
```

- Таймаут отсчитывается от создания дочернего контекста.
- Если дедлайн родителя наступает раньше, дочерний контекст его не продлевает.
- Не откладывайте cancel до конца большого цикла: ограничьте время жизни каждой операции отдельной функцией.

WithCancelCause и Cause

Фрагмент функции; Go 1.20+

```
ctx, cancel := context.WithCancelCause(parent)
defer cancel(nil)

cancel(errors.New("проверка платежа отклонена"))
fmt.Println(ctx.Err())           // context canceled
fmt.Println(context.Cause(ctx))  // причина отмены
```

- Err сообщает категорию отмены, Cause позволяет получить её предметную причину.
- До отмены Cause возвращает nil. Первой отменой фиксируется причина.

Причина истечения дедлайна или таймаута

Фрагмент; при истечении таймаута, если родитель не отменён раньше

```
cause := errors.New("сервис не уложился в лимит")
ctx, cancel := context.WithTimeoutCause(parent,
    time.Second, cause)
defer cancel()

<-ctx.Done()
fmt.Println(ctx.Err())
fmt.Println(context.Cause(ctx))
```

- WithDeadlineCause принимает время, WithTimeoutCause - длительность. Обе функции доступны с Go 1.21.
- Заданная причина используется при истечении их дедлайна. Возвращённый cancel() сам эту причину не устанавливает.

WithValue: данные одного запроса

Тип объявлен отдельно; остальные строки внутри функции

```
type requestIDKey struct{}

ctx := context.WithValue(parent, requestIDKey{}, "req-42")
id, ok := ctx.Value(requestIDKey{}).(string)
if ok {
    fmt.Println("request_id:", id)
}
```

- Передавайте request ID, trace ID и другие данные, относящиеся к запросу.
- Собственный тип ключа уменьшает риск конфликтов.
- Проверяйте тип через форму value, ok. Значения контекста не заменяют обычные аргументы функции.

Контекст в HTTP и запросах к БД

Независимые фрагменты функции, возвращающей error

```
req, err := http.NewRequestWithContext(ctx,
    http.MethodGet, url, nil)
if err != nil { return err }
resp, err := client.Do(req)
if err != nil { return err }
defer resp.Body.Close()

// В работе с БД выбирайте API с контекстом:
rows, err := db.QueryContext(ctx, query)
```

- В обработчике HTTP берите исходный контекст из r.Context().
- Проверяйте ошибки и освобождайте тело ответа или rows после успешного создания.

Группа задач: отмена при первой ошибке

Учебная реализация; каждая задача должна реагировать на ctx

```
func run(ctx context.Context, jobs []func(context.Context) error) error {
    ctx, cancel := context.WithCancel(ctx)
    defer cancel()
    results := make(chan error, len(jobs))
    for _, job := range jobs {
        go func(fn func(context.Context) error) {
            results <- fn(ctx)
        }(job)
    }
    var first error
    for range jobs {
        if err := <-results; err != nil && first == nil {
            first = err
            cancel()
        }
    }
    return first
}
```

Почему группа завершается корректно

- Результатов ровно столько, сколько запущено задач. Канал буферизирован по их числу.
- При первой полученной ошибке вызывается cancel; остальные задачи получают сигнал.
- Сбор результатов продолжается: run не возвращается раньше всех задач.
- Передача job аргументом замыкания явно фиксирует нужную функцию.
- Если задача игнорирует контекст и зависает, эта реализация не сможет завершить её принудительно.

Частые ошибки

- Потерян cancel: ресурсы остаются заняты дольше необходимого. Это не означает немедленную аварию.
- Сигнал ctx.Done() не проверяется: работа продолжается до собственного завершения.
- Для каждого внутреннего вызова создаётся Background: разрывается связь с исходным запросом.
- Через WithValue передаётся общая изменяемая map без синхронизации: Context не защищает содержимое значений.
- В Context передаётся nil вместо корректного родительского контекста.

Итог: три разных действия

- Сообщить об отмене: cancel().
- Отреагировать: select с ctx.Done() или API с поддержкой контекста.
- Дождаться завершения: канал результатов или WaitGroup.
- Для временного бюджета используйте Deadline/Timeout, для диагностики - Cause, для данных запроса - Value.
- После возврата main процесс завершается; context нужен для управляемого завершения до этого момента.

Материалы и источники

Учебная реконструкция по конспекту урока Context курса Go и найденным материалам. Исправлено объяснение жизненного цикла горутин; добавлены отсутствовавшие темы. Это новая подготовленная версия, не утраченный оригинальный PDF.

- Официальный пакет context: методы и функции отмены
- Спецификация: выполнение программы и завершение main
- Справка HTTP и database/sql: передача контекста в операции

https://pkg.go.dev/context
https://go.dev/ref/spec#Program_execution
https://pkg.go.dev/net/http
https://pkg.go.dev/database/sql